Audspect BAS — Monitoring and Logging Guide

Classification: Internal — Audspect Engineering / Confidential
Platform Version: v1.7.5

Overview

Audspect BAS produces logs at three levels: structured application logs from the orchestrator, PostgreSQL query logs, and Docker container-level logs. There is no built-in log shipping — export to your SIEM or log aggregator via Docker logging drivers.

Orchestrator Log Format

The orchestrator logs to stdout (structured log lines):

```
2026-07-01T09:00:00Z [INFO] agent connected agent_id=agt-
2026-07-01T09:01:00Z [WARN] scenario signature invalid sc
2026-07-01T09:02:00Z [ERROR] run dispatch failed run_id=r
```

There is no configurable log level. The orchestrator uses Go's standard `log` package throughout with ad-hoc bracketed prefixes (`[+]`, `[!]`, `[connector]`, `[svc]`, etc., varying by subsystem) rather than a strict `LEVEL key=value` convention — the log lines below are illustrative of the kind of information logged, not a literal contract on exact wording or format. There is no `BAS_LOG_LEVEL` env var;

everything logs unconditionally.

Key Log Events

Authentication

```
[INFO] user login success username=analyst@company.com ip
[WARN] user login failed username=unknown@company.com ip=
[WARN] rate limit exceeded ip=192.168.1.50 endpoint=/api/
```

Agent Lifecycle

```
[INFO] agent enrolled agent_id=agt-new hostname=WIN-NEW i
[INFO] agent heartbeat agent_id=agt-abc123 binary_trust=t
[WARN] agent binary trust failed agent_id=agt-abc123 hash
[INFO] agent state changed agent_id=agt-abc123 old=active
```

Scenario Execution

```
[INFO] run dispatched run_id=run-xyz scenario_id=safe-sim
[INFO] run step result run_id=run-xyz step_id=step-001 ve
[INFO] run completed run_id=run-xyz prevention_score=82 e
[ERROR] run delivery failed run_id=run-xyz agent_id=agt-a
```

Integrity

```
[WARN] scenario signature verification failed scenario_id
[WARN] filesystem tamper detected scenario_id=my-scenario
[INFO] scenario loaded with valid signature scenario_id=s
```

Attack Path

```
[INFO] ap job created job_id=ap-job-xyz agent_id=agt-abc1
[INFO] ap job dispatched job_id=ap-job-xyz
[INFO] ap job stage updated job_id=ap-job-xyz stage=probi
[INFO] ap job completed job_id=ap-job-xyz nodes=52 edges=
[WARN] ap job delivery failed job_id=ap-job-xyz reason=ac
```

Viewing Logs

Docker Compose

```
# All services, real-time
docker compose logs -f

# Orchestrator only, last 200 lines
docker compose logs orchestrator --tail=200

# Filter for errors
docker compose logs orchestrator | grep "\[ERROR\]\|\[WAR

# Filter for a specific agent
docker compose logs orchestrator | grep "agent_id=agt-abc
```

Searching logs

```
# All signature failures today
docker compose logs orchestrator | grep "signature\|tampe

# All failed logins
docker compose logs orchestrator | grep "login failed"
```

```
# All binary trust issues
docker compose logs orchestrator | grep "binary trust"
```

Platform Health Monitoring

Health endpoint

```
# Check platform health
curl -s http://localhost:9443/health

# Expected response shape
{"status":"ok","db":"ok","version":"<current version>"}
```

Responses:

- `200 {"status":"ok"}` — healthy
- `503 {"status":"degraded","db":"unavailable"}` — database unreachable

Container health

```
docker compose ps
```

Watch for containers in `Exit` or `Restarting` state.

PostgreSQL metrics

```
# Active connections
docker compose exec postgres psql -U bas_user -d bas_plat
  -c "SELECT count(*) FROM pg_stat_activity WHERE state='
```

```
# Table sizes
docker compose exec postgres psql -U bas_user -d bas_plat
  -c "SELECT schemaname, tablename, pg_size_pretty(pg_tot

# Long-running queries
docker compose exec postgres psql -U bas_user -d bas_plat
  -c "SELECT pid, now() - pg_stat_activity.query_start AS
```

Disk Usage Monitoring

```
# Docker volume usage
docker system df -v

# audspect-postgres-data volume specifically
docker run --rm -v audspect-postgres-data:/data alpine du
```

There's no equivalent command for the orchestrator container's own disk usage — it runs on a `distroless` base image with no shell, so `docker exec`/`docker compose exec` into it doesn't work at all (`OCI runtime exec failed: exec: "df": executable file not found`). Its filesystem is effectively read-only application code plus bundled content anyway; there's nothing meaningful to grow there between releases. Check the host's own disk usage under Docker's data root instead if orchestrator-image size is a concern.

Alert thresholds:

- PostgreSQL data directory > 80% full: schedule maintenance
- PostgreSQL data directory > 90% full: immediate action required (stop ingestion, archive old data)

Audit Log API

The platform's audit log (user-facing, `CanViewAuditLogs`) is accessible via:

```
GET /api/audit-logs?limit=100&offset=0&action=<exact acti
```

- `limit` — max rows, capped at 500 (default 100)
- `offset` — pagination offset
- `action` — exact match against the recorded action string (e.g. `connector.config.update`, `openaev.sync`, `user.create` — dotted-name strings scoped per feature, not a fixed enum of categories; grep `h.auditLog(` / `h.auditLogAs(` calls across `internal/api/` for the current full list)
- `actor` — case-insensitive substring match against the acting username

There is no separate CSV export endpoint or `category`/`from`/`to` date-range filter — date filtering isn't supported server-side today; page through `limit`/`offset` and filter client-side if a date range is needed.

Exporting Logs to a SIEM

Docker syslog driver

```
services:
  orchestrator:
    logging:
      driver: syslog
      options:
        syslog-address: "tcp://siem.company.com:514"
        tag: "audspect-orchestrator"
```

Docker fluentd driver

```
services:
  orchestrator:
    logging:
      driver: fluentd
      options:
        fluentd-address: "localhost:24224"
        tag: "audspect.orchestrator"
```

JSON file driver (default) + Filebeat

Default Docker logging writes JSON to `/var/lib/docker/containers/<id>/<id>-json.log`. Configure Filebeat to ship these to Elasticsearch.

Key Metrics for SOC Dashboards

Metric	Query	Alert Threshold
Failed logins per hour	`grep "login failed"	count`
Agent binary trust failures	`grep "binary trust failed"`	Any
Scenario signature failures	`grep "signature verification failed"`	Any
Agent offline count	`GET /api/agents` (no server-side state filter — online/offline/degraded bucketing is computed client-side from `status`/`state`, count locally or via a direct SQL query)	>20% of fleet

Database connection errors	`grep "db.*error"`	Any
AP job delivery failures	`grep "ap job delivery failed"`	>3/day

© Audspect Engineering — Internal / Confidential